Department of Computer Science, University College London

COMP0222/COMP0249 Lab 30: RGBD Camera

10th Februray, 2026 ver. 20260210

1 Scope and Purpose

In this lab, you will explore 3D spatial data using the Intel RealSense D455. The D455 uses stereoscopic vision to provide a dense 3D point cloud. You will implement a data logging framework to capture synchronized RGB and Depth frames, ensuring they are spatially aligned.

You will distinguish between raw sensor data and processed metadata, utilizing the camera's global shutter to capture motion-blur-free imagery. Finally, you will explore the fundamentals of 3D reconstruction by preparing your captured data for a Structure-from-Motion (SfM) pipeline using COLMAP. This enables building full 3D digital models.

2 Scenario

Motion and mapping in 3D space are characterized by 6 Degrees of Freedom (DoF), consisting of translation (x, y, z) and rotation (roll, pitch, yaw). In this lab, you will perform data logging to build a dataset for 3D reconstruction. 3D sensing allows for the detection of complex geometries, such as table surfaces, hanging obstacles, and terrain irregularities. By aligning the depth map with the RGB sensor, you create a "RGB-D" image where every color pixel is associated with a real-world distance. This technique is fundamental for spatial computing, robotic manipulation, and large-scale architectural scanning.

3 Sensor Specification: Intel RealSense D455

The Intel RealSense D455 is a stereoscopic depth camera designed for long-range and high-precision applications, see Fig. 1. It operates based on the principle of Stereo Vision with Infrared Projector assistance.

- **Mechanism: Two global shutter sensors (separated by a 95mm baseline) calculate depth via disparity. An infrared projector helps the sensors "see" texture on flat, featureless surfaces (like white walls).**
- **Synchronization: An internal hardware clock ensures that the RGB and Depth frames are captured at the exact same moment.**
- **Key Specifications:**
  - **Ideal Range: 0.6m – 6m (Depth can reach up to 20m)**
  - **Sensor Type: Global Shutter (minimizes motion blur)**
  - **Field of View (H $\times$ V):** $86^\circ \times 57^\circ$
  - **RGB Resolution: Up to** 1280×800 **at 30fps**
  - **Connection: USB 3.1 Gen 1 (Type-C)**

Figure 1: Intel RealSense D455 Depth Camera.

4 Requirements

This lab requires Python 3.8+. You must create a virtual environment to manage hardware-specific libraries. The RealSense SDK (`pyrealsense2`) is highly sensitive to binary compatibility; using a `venv` prevents conflicts with system-level packages (see the next section for venv).
Note: Ensure your computer is equipped with a USB 3.0/3.1 port. A USB 2.0 port will severely limit resolution and framerate.

5 Activities

Activity 0: Basic Connectivity and Environment Setup

- **Objective: Establish communication with the Intel RealSense D455 and isolate the development environment to ensure library compatibility.**
- **Instructions:**
  - **Step 1: Driver and SDK Installation**
    - **Windows: Download the latest** `Intel.RealSense.SDK-Win10.exe` **from the Official GitHub Releases: https://github.com/IntelRealSense/librealsense/releases/latest. Run the installer to set up the RealSense Viewer and USB drivers.**
    - **Ubuntu: Register the Intel repository and install via** `apt` **using the following commands:**

```
sudo apt install librealsense2-dkms librealsense2-utils
sudo apt install librealsense2-dev librealsense2-dbg
```

  - **Step 2: Firmware Verification Launch the** `RealSense Viewer`**. If a "Firmware Update Available" notification appears in the top-right corner, proceed with the update to ensure the D455's global shutter and IMU are properly calibrated.**
  - **Step 3: Virtual Environment Setup Open your project folder and initialize the Python environment:**

```
python -m venv rs_env
source rs_env/bin/activate  # Linux/Mac
rs_env\Scripts\activate     # Windows
pip install pyrealsense2 opencv-python numpy
```

Once the viewer is installed, run it (see Fig. 2). Explore the 2D and 3D options, see the top right corner.

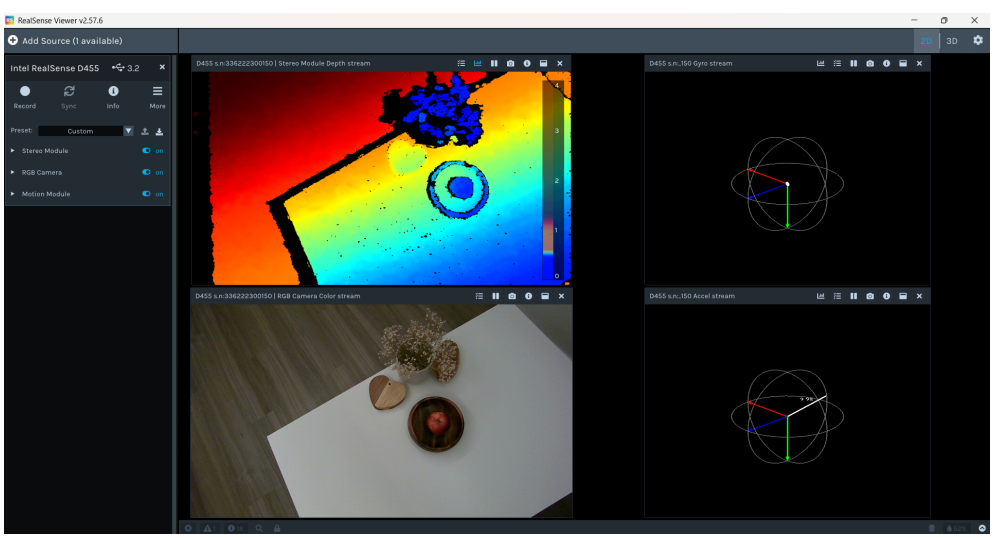

Figure 2: Intel D455 viewer.

Activity 1: Real-Time Aligned Data Logging

- **Objective: Capture synchronized RGB-D pairs where the depth map is aligned to the RGB camera's coordinate system.**
- **Instructions:**
  - **Run the** `d455_logger.py` **script.**
  - **This script uses the** `rs.align` **object to mathematically project the depth data onto the RGB lens's viewpoint.**
  - **Press SPACE to save a "Raw" frame.**
- **Data Output:**
  - `.png`**: Lossless RGB image.**
  - `.npy`**: Raw 16-bit NumPy array containing depth in millimeters.**
  - `.csv`**: Metadata log containing hardware exposure and gain values.**

Activity 2: Data Playback and Quality Inspection

- **Objective: Review the logged 3D data and verify that the RGB-Depth alignment is correct.**
- **Instructions:**
  - **Run the** `d455_browser.py` **script.**
  - **Use SPACE to iterate through frames or 'c' for continuous playback.**
  - **Observe the "Depth Shadows", i.e. areas where the camera could not see (represented as black pixels). Ensure your target objects are not within these shadow regions.**

Activity 3: Dataset Preparation for 3D Reconstruction (COLMAP)

- **Objective: Export captured data for processing in an external Structure-from-Motion (SfM) pipeline.**
- **Instructions:**
  - **Isolate the** `.png` **files into an** `/images` **directory.**
  - **Load the directory into COLMAP.**
  - **Perform Feature Extraction and Sequential Matching.**
  - **Because the D455 uses a global shutter, features will remain sharp even if the camera was moving during capture, resulting in a higher-quality 3D point cloud.**
  - **Try two sets of data: a small scale from a desktop object, a large scale environment, e.g. a room-sized environment.**